

Julia Reader

An In-Depth White Paper for an Open Research Harness in Agentic Reading

A designed PDF edition for builders and researchers who need to understand how Julia Reader works internally: the core loop, the reading lake, Chronicle generation, packet handoff, and validation.

RAG retrieves. Summarizers compress. Julia Reader reads, records, reconstructs, and validates.

VERSION

0.2 expanded draft

PROJECT TYPE

Open-source research harness

PRIMARY LANGUAGE

Python

CORE OUTPUT

Chronicle-based Markdown books, sentence maps, reading lake metadata, live understanding state, validation packets

PRIMARY AUDIENCE

builders, researchers, AI engineers, agent developers, knowledge-management teams, educators, and reviewers who need to understand how Julia Reader works internally

CORE READING LOOP

Source

Sentence Map

Bounded Chunks

Reading Lake

Chronicle

Packet

Validation

Contents

A navigable map of the reading protocol, evidence lake, Chronicle artifact, validation layer, and open research agenda.

Abstract	5
Executive Summary	5
1. The Problem: Long Context Is Not the Same as Reading	6
2. Existing Approaches and Where Julia Reader Fits	7
2.1 Retrieval-Augmented Generation	7
2.2 Chunking and Hierarchical Indexing	8
2.3 Map-Reduce and Refinement Summarization	8
2.4 GraphRAG and Structured Understanding	9
2.5 Agent Frameworks	9
3. Core Thesis	10
4. What Julia Reader Produces	10
5. Key Terms	12
5.1 Agentic Reading	12
5.2 Chronicle	12
5.3 Lake	12
5.4 Sentence Map	13
5.5 Bounded Chunk	13
5.6 Live Understanding	13
5.7 Packet	13
5.8 Validation	13
6. System Overview	13
7. The Core Loop	14
8. Source Loading and Normalization	16
8.1 Source Anchors	17
9. Sentence Maps	18
9.1 Why Sentence-Level Mapping Matters	18
9.2 Deterministic First, Model-Backed Later	19
10. Bounded Chunks	19
10.1 Chunking Goals	20
10.2 Chunk Boundary Algorithm	20
10.3 Overlap	21
11. Semantic Break Detection	22
11.1 Deterministic Signals	22
11.2 Model-Backed Signals	22
11.3 Why Breaks Matter	23
12. Live Understanding	23
12.1 Why Live Understanding Exists	24
12.2 Live Understanding Is Not Hidden Chain of Thought	24
12.3 Update Contract	25
13. The Lake	25
13.1 Lake Design Principles	26
13.2 Lake Folder Layout	26
13.3 Lake Record Types	28
13.4 The Lake as a Run Log	29
13.5 The Lake Is Not the Chronicle	30

13.6 Lake and Resumability	30
13.7 Lake and Human Review	31
14. Local Reading	31
14.1 Local Reader Prompt Contract.....	32
14.2 Local Read Is Not Final Synthesis	32
15. Live Synthesis and State Updates	32
15.1 State Growth Control	33
15.2 State Versioning	33
16. Chronicle Generation	34
16.1 Chronicle Is Reconstruction, Not Compression Alone	34
16.2 Building the Outline	34
16.3 Chapter Rendering	35
16.4 Multiple Chronicle Styles	35
17. Reader Packet	36
17.1 Packet Design Goals	37
17.2 Packet Use Cases	37
18. Validation	37
18.1 Validation Questions	38
18.2 Coverage Validation	38
18.3 Claim Support Validation	38
18.4 Contradiction and Tension Detection	39
18.5 Validation Report	39
19. Deterministic and Model-Backed Limbs	40
19.1 Deterministic Limbs	40
19.2 Model-Backed Limbs	41
19.3 Why the Distinction Matters	41
20. Prompt and Schema Contracts.....	41
20.1 Why Schemas Matter.....	42
20.2 Prompt Replacement	42
21. Failure Modes	42
21.1 Sentence Splitting Errors	42
21.2 Bad Chunk Boundaries	43
21.3 Over-Compression	43
21.4 State Drift.....	43
21.5 Unsupported Chronicle Claims	43
21.6 False Confidence	43
21.7 Model Variance.....	43
21.8 Source Extraction Errors	43
21.9 Privacy and Sensitive Data.....	43
22. Evaluation Strategy	43
22.1 Source Coverage	43
22.2 Claim Traceability	44
22.3 Summary Faithfulness	44
22.4 Middle Retention.....	44
22.5 Human Review	44
22.6 Downstream Usefulness	44
22.7 Resumability and Robustness.....	45
22.8 Latency and Cost	45
23. Example Use Case: Transcript to Chronicle	45
24. Example Use Case: Research Paper to Study Chronicle.....	46
25. Example Use Case: Policy Document to Compliance Packet	47
26. Repository Structure	47
27. CLI Sketch	49

28. Open-Source Philosophy	49
29. Roadmap	50
Phase 1: Public Research Release	50
Phase 2: Reading Loop Stability	50
Phase 3: Chronicle System	51
Phase 4: Validation	51
Phase 5: Research Extensions	51
Phase 6: Agent Integrations	51
30. What Julia Reader Is Not	52
31. Why This Matters	52
31.1 For Agents	52
31.2 For RAG	52
31.3 For Compliance and Legal Workflows	53
31.4 For Education	53
31.5 For Research	53
32. Short Public Thesis	54
33. Conclusion	54
Appendix A: Minimal Data Schemas	55
A.1 Sentence	55
A.2 Chunk	55
A.3 Reading Event	56
A.4 Claim	56
A.5 Packet	56
Appendix B: Example Validation Rules	57
Appendix C: Recommended README Positioning	57
Appendix D: Recommended Next Step for the Repo	58
References	58

● Abstract

Large language models can now accept increasingly long context windows, but long context alone does not guarantee reliable reading. Research on long-context behavior has shown that models may still struggle to use information robustly when relevant material appears in the middle of long inputs, a pattern commonly described as the “lost in the middle” problem. In that work, model performance was often strongest when relevant material appeared near the beginning or end of the context and weaker when it appeared in the middle. [1](#)

Julia Reader is an experimental open-source harness for **agentic reading**. It treats reading not as a single prompt, flat summary, retrieval call, or context-window exercise, but as a staged loop: source loading, normalization, sentence mapping, bounded chunking, break detection, local reading, live synthesis, evidence storage in a lake, Chronicle generation, packet creation, and validation.

The project explores a simple research question:

Can long-form machine reading become more reliable, auditable, and useful if it is processed as a progressive, inspectable loop instead of a single flat completion?

Julia Reader does not claim that an LLM “understands” a text in the human sense. Instead, it provides a structured protocol for helping machines **read, gather, integrate, organize, reconstruct, and check** long-form material in a way that humans and downstream agents can inspect.

The key idea is that the output of a reading system should not only be an answer. It should also include a record of how the answer became possible: what was read, how it was divided, where each claim came from, what changed in the running understanding, what was omitted, and what still requires review.

● Executive Summary

Julia Reader is a research harness for turning long source material into durable reading artifacts.

It starts with raw material such as a transcript, article, policy document, research paper, book excerpt, code explanation, or meeting record. It then builds a sentence-level map of that source, groups the sentences into bounded semantic chunks, processes each chunk through a reading loop, saves evidence into a structured **lake**, updates a running **live understanding**, and reconstructs the result into a human-browsable **Chronicle**.

The Chronicle is not intended to be only a summary. It is a book-like object that can contain chapters, source-aware sections, claim maps, open questions, entities, decisions, themes, and validation results.

The lake is the internal evidence layer. It stores sentence IDs, chunk IDs, break signals, topic labels, local summaries, live-understanding history, claim support, generated outline mappings, packet fields, and

validation data. The lake exists so the Chronicle does not become a floating narrative detached from its source.

Julia Reader's core loop is:

```
load source
→ normalize source
→ create sentence map
→ create bounded chunks
→ detect semantic breaks
→ read each chunk
→ update live understanding
→ write evidence to lake
→ build Chronicle outline
→ render Markdown Chronicle
→ create downstream packet
→ validate coverage and support
```

This loop makes long-form reading slower, more explicit, and more inspectable. That is the point. Julia Reader is not trying to make the model appear smarter through a single large prompt. It is trying to create a repeatable protocol that exposes the mechanics of reading.

In short:

RAG retrieves. Summarizers compress. Julia Reader reads, records, reconstructs, and validates.

1 The Problem: Long Context Is Not the Same as Reading

The AI industry has often treated long-form document understanding as a context-window problem. If a model can accept more tokens, the assumption is that it can read more effectively. In practice, this is only partly true.

Long context windows are useful. They reduce the need to cut everything down before the model sees it. They allow more of the original material to stay available in a single call. They can also improve workflows that previously required many brittle summarization steps.

But long context alone does not solve reading.

Even with large windows, long documents introduce persistent problems:

1. **Middle loss** — important information may be ignored or underweighted when it is buried in the middle of a long context.

2. **Drift** — the model may gradually lose track of the original purpose, definitions, constraints, or narrative structure.
3. **Compression loss** — summarization often removes details that later become important.
4. **Opaque processing** — many systems hide their chunking, summarization, reasoning, source selection, or synthesis steps.
5. **Weak source navigability** — outputs often cannot be traced back to a sentence, paragraph, page, timestamp, or transcript moment.
6. **One-shot illusion** — a single completion can look fluent while failing to represent the full document faithfully.
7. **Poor recovery** — when a run fails, many systems cannot resume cleanly at the last known good reading step.
8. **Weak auditability** — a reviewer may see the final answer but not the source coverage or intermediate transformations that produced it.

The “Lost in the Middle” line of work is important because it challenges the assumption that a long-context model reliably uses all parts of its context equally. The problem is not merely whether the model can accept a long input. The problem is whether the model can use the right information from the right place at the right time. [1](#)

Julia Reader begins from this premise:

A model call is not a reading process. A model call is one operation inside a reading process.

The project therefore treats long-form reading as an explicit pipeline with a state, a memory, an audit trail, and a final artifact.

2 Existing Approaches and Where Julia Reader Fits

Julia Reader sits near several existing fields: retrieval-augmented generation, chunking, long-document summarization, hierarchical indexing, graph-based retrieval, and multi-agent workflows. It does not claim to replace these approaches. Instead, it defines a narrower and more concrete problem: how an agent should read long material and leave behind an inspectable artifact of that reading.

2.1 Retrieval-Augmented Generation

Retrieval-augmented generation, or RAG, connects a generative model to external memory. In the original formulation, a model combines parametric memory with a non-parametric memory, such as a dense vector index, and retrieves passages that help answer a task. [3](#)

Modern RAG systems commonly split source documents into chunks, embed those chunks, retrieve relevant passages, and send those passages to a model. This is powerful, especially when the user has a query.

RAG usually asks:

“What pieces of this corpus are relevant to this question?”

Julia Reader asks:

“How can an agent read this material progressively, preserve the path of reading, and leave behind an inspectable record?”

RAG is often query-first. Julia Reader is reading-first.

A Julia Reader output can later feed a RAG system. The Chronicle, sentence map, chapter map, claim map, and packet can all become better inputs for indexing and retrieval. But Julia Reader’s primary goal is not retrieval. Its primary goal is structured reading.

2.2 Chunking and Hierarchical Indexing

Chunking is a necessary part of most long-document LLM workflows. Frameworks such as LlamaIndex support hierarchical node parsing, where input is split into multiple levels of nodes, including parent and child chunks at different sizes. [4](#)

Julia Reader uses chunking too, but it treats chunking as only one part of the reading loop.

A chunk in Julia Reader is not merely a retrievable text span. It is a bounded reading unit with:

- a source sentence range,
- token estimates,
- topic and break metadata,
- local summary fields,
- extracted entities and claims,
- relationship to previous and next chunks,
- and a place in the live-understanding update history.

This means a Julia Reader chunk is both a text segment and an event in a reading sequence.

2.3 Map-Reduce and Refinement Summarization

Long-document summarization often uses map-reduce or refinement patterns. In a map-reduce pattern, smaller sections are summarized first, and then those summaries are combined. LangChain documentation describes map-reduce-style document combination as mapping a chain over documents and then reducing the results. [5](#)

Julia Reader overlaps with refinement summarization because it keeps a running state. But the goal is broader than compression.

A summarizer asks:

“How do we make this shorter?”

Julia Reader asks:

“How do we make this readable, inspectable, reconstructable, and useful downstream?”

The difference matters. A summary may omit source locations. A Chronicle should preserve them. A summary may hide intermediate state. Julia Reader stores the state. A summary may collapse disagreement or uncertainty. Julia Reader can preserve open questions, contradictions, warnings, and validation status.

2.4 GraphRAG and Structured Understanding

GraphRAG is an important adjacent system. Microsoft’s GraphRAG describes a structured, hierarchical approach to RAG that extracts a knowledge graph from raw text, builds community hierarchies, generates summaries for those communities, and uses those structures in retrieval-augmented tasks. [6](#)

Julia Reader is not trying to replace GraphRAG. GraphRAG is strongly oriented toward graph-based discovery and retrieval. Julia Reader is oriented toward the act of reading itself: how material is encountered, segmented, synthesized, preserved, reconstructed, and checked.

The two can complement each other. Julia Reader can produce structured evidence that a graph pipeline may consume. A graph system can enrich a Chronicle with entity relationships, communities, and cross-document connections. But the abstractions are different:

GraphRAG: corpus → graph → community summaries → graph-aware retrieval
 Julia Reader: source → reading loop → lake → Chronicle → validation packet

2.5 Agent Frameworks

Agent frameworks such as AutoGen support event-driven and multi-agent AI workflows. AutoGen describes itself as an event-driven framework for scalable multi-agent AI systems. [7](#)

Julia Reader can integrate with agent systems, but it is not a general agent framework. It is a reading harness. It answers a specific operational question:

What should an agent do before it acts on a long source?

Julia Reader’s answer is:

It should create a structured reading artifact, preserve source evidence, and validate the artifact before downstream use.

3 Core Thesis

Julia Reader is based on the belief that machine reading should be treated as a **protocol**, not only a prompt.

The central thesis is:

Agents read better when long-form input is processed as a progressive, inspectable loop instead of a single prompt, flat summary, or hidden RAG index.

This creates a distinction between four common approaches:

Approach	Primary Goal	Typical Strength	Typical Limitation
Long-context prompting	Put everything into one model call	Simple, direct, fewer pipeline pieces	Can still drift or miss buried information
Summarization	Compress the source	Produces a shorter representation	May lose structure, evidence, and nuance
RAG	Retrieve relevant chunks	Good for query answering over external memory	Optimized for questions, not full reading
Julia Reader	Read, record, reconstruct, validate	Preserves process and source traceability	More complex, slower, still experimental

Julia Reader’s core claim is not that one loop eliminates all error. It does not. The claim is that a loop with explicit state, evidence, source IDs, validation, and durable artifacts creates a better surface for inspecting and improving machine reading.

4 What Julia Reader Produces

Julia Reader turns long-form source material into a structured **Chronicle** backed by a reading lake.

The current and planned outputs are:

1. **Source manifest** — a record of the source file, source type, ingestion time, hash, title, and run configuration.
2. **Sentence map** — a sentence-level representation of the source.
3. **Chunk map** — a token-bounded and semantically aware grouping of sentences.
4. **Break map** — topic-shift and boundary signals.
5. **Reading events** — records of each chunk-level reading operation.
6. **Live understanding history** — the evolving state of what the system believes has been established.
7. **Claim map** — generated claims linked back to source sentences or chunks.
8. **Entity and theme maps** — people, organizations, concepts, topics, and recurring ideas.
9. **Chronicle outline** — the generated book-like structure.
10. **Markdown Chronicle** — human-browsable chapters and index files.
11. **Reader packet** — a compact downstream handoff object.
12. **Validation report** — coverage, traceability, contradiction, and warning checks.

A typical output folder may look like this:

```

chronicles/
  chronicle_2026_05_09_001/
    manifest.json

    source/
      source.txt
      source_metadata.json

    lake/
      sentence_map.jsonl
      chunk_map.jsonl
      break_map.jsonl
      reading_events.jsonl
      chunk_summaries.jsonl
      live_understanding_history.jsonl
      live_understanding_latest.json
      entities.jsonl
      themes.jsonl
      claims.jsonl
      claim_support.jsonl
      chapter_map.jsonl
      coverage.json

    chronicle/
      index.md
      chapters/
        01-opening-context.md
        02-core-argument.md
        03-technical-architecture.md
        04-open-questions.md

    packets/
      reader_packet.json

```

```
validation/  
  validation_report.md  
  validation_report.json
```

The exact folder layout can evolve. The principle should not: generated artifacts must remain connected to source evidence.

5 Key Terms

5.1 Agentic Reading

Agentic reading is the process by which an AI system works through source material in a staged, stateful, tool-using way rather than simply receiving the whole source as one prompt.

In Julia Reader, the agent does not merely answer. It performs a reading procedure:

```
observe source  
segment source  
read bounded unit  
record evidence  
update state  
repeat  
reconstruct  
validate
```

5.2 Chronicle

A Chronicle is the human-browsable artifact generated from the reading process. It is usually Markdown-based. It can read like a short book, guide, study note, technical brief, or research digest.

A Chronicle should be understandable to humans, useful to agents, and traceable back to source records.

5.3 Lake

The lake is the structured storage layer for reading evidence. It is not just a dump of files. It is a run-scoped record of what the system read, how it segmented the source, what it concluded at each step, how the live understanding changed, and how final outputs map back to the source.

A simple definition:

The lake is the memory of the reading run.

5.4 Sentence Map

A sentence map is a structured representation of the source at sentence level. Each sentence receives a stable ID and metadata.

5.5 Bounded Chunk

A bounded chunk is a controlled unit of reading. It contains one or more sentences, stays below a configured token limit, and preserves source sequence.

5.6 Live Understanding

Live understanding is the running synthesis that updates as each chunk is processed. It records what has been established, what changed, what remains uncertain, and what should be watched for in later chunks.

5.7 Packet

A packet is a downstream handoff object. It contains a compact representation of the Chronicle and lake outputs for use by other agents, RAG systems, compliance tools, education tools, report generators, or internal knowledge systems.

5.8 Validation

Validation is the discipline layer. It checks whether the system processed the source, preserved traceability, avoided unsupported claims, and surfaced uncertainty instead of hiding it.

6 System Overview

Julia Reader can be understood as five cooperating layers:

1. Source layer
Raw input, normalized text, source metadata, source hash.
2. Mapping layer
Sentence map, token estimates, source locations, paragraph/page/timestamp anchors.
3. Reading layer
Bounded chunks, semantic break detection, model-backed local reading, live understanding updates.
4. Lake layer
Evidence storage, event logs, summaries, claims, entity maps, theme maps, state history.
5. Artifact layer
Chronicle, packet, validation report, export formats.

The layers are intentionally separated. Sentence splitting should not depend on chapter generation. Validation should not require rerunning the model. The Chronicle should not be the only place where evidence lives.

A high-level flow:



The model is not the system. The model is used inside the system.

7 The Core Loop

The core loop is the heart of Julia Reader.

A simplified version:

1. Load the source.
2. Normalize the source.
3. Split the source into sentences.
4. Assign stable sentence IDs.
5. Estimate tokens per sentence.
6. Build bounded chunks from sentence ranges.
7. Detect semantic breaks and topic shifts.
8. Initialize live understanding.
9. For each chunk:
 - a. read the chunk,
 - b. summarize local content,
 - c. extract claims, entities, themes, and open questions,
 - d. compare the chunk to the current live understanding,
 - e. update live understanding,
 - f. write the reading event into the lake.
10. Build a Chronicle outline from the accumulated lake evidence.
11. Generate Markdown pages.
12. Create a downstream packet.
13. Run validation.

A more technical pseudocode version:

```
def run_julia_reader(source_path: str, config: ReaderConfig) -> ChronicleRun:
    run = create_run(config=config)

    source = load_source(source_path)
    normalized = normalize_source(source)
    write_source_manifest(run, source, normalized)

    sentences = build_sentence_map(normalized, config)
    lake.write_jsonl("sentence_map", sentences)

    chunks = build_bounded_chunks(sentences, config)
    lake.write_jsonl("chunk_map", chunks)

    breaks = detect_breaks(chunks, sentences, config)
    lake.write_jsonl("break_map", breaks)

    live = initialize_live_understanding(source_metadata=source.metadata)
    lake.write_json("live_understanding_latest", live)

    for chunk in chunks:
        local_result = read_chunk(
            chunk=chunk,
            sentences=sentences,
            live_understanding=live,
            config=config,
        )

        event = create_reading_event(chunk, local_result, live)
        lake.append_jsonl("reading_events", event)
        lake.append_jsonl("chunk_summaries", local_result.summary_record)
        lake.append_jsonl("claims", local_result.claims)
        lake.append_jsonl("entities", local_result.entities)
        lake.append_jsonl("themes", local_result.themes)
```

```
live = update_live_understanding(live, local_result)
lake.append_jsonl("live_understanding_history", live)
lake.write_json("live_understanding_latest", live)

outline = build_chronicle_outline(lake, config)
lake.write_jsonl("chapter_map", outline.chapter_map)

chronicle = render_chronicle(outline, lake, config)
packet = build_reader_packet(chronicle, lake, config)

validation = validate_run(run, lake, chronicle, packet, config)

return ChronicleRun(
    run_id=run.id,
    chronicle=chronicle,
    packet=packet,
    validation=validation,
)
```

This loop is intentionally boring in the best way. It makes each stage explicit. It gives contributors places to improve behavior without rewriting the whole system.

8

Source Loading and Normalization

The source layer is responsible for turning input into stable text.

Supported or planned source types may include:

- plain text,
- Markdown,
- transcripts,
- PDFs after extraction,
- HTML after cleaning,
- code documentation,
- meeting notes,
- email threads,
- policy documents,
- books or chapters,
- pasted chat logs.

The loader should produce a normalized source object:


```
{
  "source_id": "src_2026_05_09_001",
  "source_title": "Meeting Transcript",
  "source_type": "transcript",
  "input_path": "examples/meeting.txt",
  "normalized_text_path": "source/source.txt",
  "source_hash": "sha256:...",
  "created_at": "2026-05-09T23:00:00Z",
  "metadata": {
    "author": null,
    "date": null,
    "language": "en",
    "estimated_tokens": 48231
  }
}
```

Normalization may include:

- standardizing line endings,
- preserving paragraph breaks,
- removing extraction artifacts when safe,
- keeping timestamps or page anchors,
- preserving speaker names in transcripts,
- recording source hashes,
- recording whether the source has been transformed.

The loader should avoid pretending that cleaned text is identical to raw input. If the system removes headers, fixes hyphenation, or normalizes whitespace, those transformations should be recorded.

8.1 Source Anchors

When possible, Julia Reader should preserve anchors such as:

- page number,
- paragraph number,
- line number,
- transcript timestamp,
- speaker label,
- section heading,
- character offset.

These anchors make later validation and review much easier.

Example sentence with anchors:

```
{
  "sentence_id": "s_000421",
  "text": "The project treats reading as an agent loop.",
  "position": 421,
  "char_start": 18492,
  "char_end": 18539,
  "page": null,
  "timestamp_start": "00:14:32",
  "timestamp_end": "00:14:36",
  "speaker": "Avery",
  "paragraph_id": "p_0081"
}
```

9 Sentence Maps

A sentence map is the first major structural artifact.

It gives every sentence a stable identity. That identity becomes the basis for chunks, claims, chapters, and validation.

A sentence map record can look like this:

```
{
  "sentence_id": "s_000421",
  "source_id": "src_2026_05_09_001",
  "text": "The project treats reading as an agent loop.",
  "position": 421,
  "paragraph_id": "p_0081",
  "char_start": 18492,
  "char_end": 18539,
  "estimated_tokens": 11,
  "section_hint": "Architecture",
  "speaker": null,
  "timestamp_start": null,
  "timestamp_end": null,
  "page": null,
  "flags": []
}
```

9.1 Why Sentence–Level Mapping Matters

Chunk–level traceability is useful, but it is often too coarse.

A generated claim may depend on one sentence inside a 1,500–token chunk. If the system only stores chunk IDs, a reviewer may need to reread the entire chunk. Sentence IDs make review more precise.

Sentence maps help with:

- source traceability,

- claim support,
- coverage accounting,
- chapter mapping,
- transcript navigation,
- recovery after failed runs,
- fine-grained validation,
- exporting to other tools.

9.2 Deterministic First, Model-Backed Later

Sentence splitting should be as deterministic as possible. Rule-based sentence boundary systems, such as spaCy's Sentencizer, provide one way to implement sentence segmentation without requiring a full statistical parser. [8](#)

However, sentence splitting can be messy. Legal documents, transcripts, bullet points, headings, code blocks, abbreviations, and OCR artifacts can all cause boundary errors. Julia Reader should therefore treat sentence maps as inspectable and correctable.

The system can support multiple sentence strategies:

```
basic_regex
spacy_sentencizer
transcript_aware
markdown_aware
pdf_extraction_aware
model_assisted_cleanup
```

The important requirement is not that one splitter is perfect. The important requirement is that the chosen strategy is recorded in the run manifest.

10 Bounded Chunks

A bounded chunk is a controlled unit of reading.

Julia Reader does not treat chunks as arbitrary slices. The goal is to respect semantic breaks while staying under a token limit.

A good chunk should be:

- small enough for reliable model processing,
- large enough to preserve local meaning,
- traceable to original sentence IDs,
- connected to adjacent chunks,
- compatible with recovery and validation,

- and easy to inspect by a human reviewer.

A chunk record can look like this:

```
{
  "chunk_id": "chunk_018",
  "source_id": "src_2026_05_09_001",
  "sentence_start": "s_000401",
  "sentence_end": "s_000432",
  "sentence_ids": ["s_000401", "s_000402", "...", "s_000432"],
  "position": 18,
  "estimated_tokens": 742,
  "char_start": 17720,
  "char_end": 19044,
  "heading_hint": "Architecture",
  "previous_chunk_id": "chunk_017",
  "next_chunk_id": "chunk_019",
  "break_before": true,
  "break_after": false,
  "topic_tags": ["agentic-reading", "architecture"],
  "status": "ready"
}
```

10.1 Chunking Goals

Chunking has several competing goals:

1. **Reliability** — keep the unit small enough for focused reading.
2. **Coherence** — avoid splitting an idea in half.
3. **Coverage** — ensure every sentence belongs to a chunk.
4. **Traceability** — preserve source sentence IDs.
5. **Throughput** — avoid creating excessive model calls.
6. **Repairability** — make failed chunks easy to rerun.

The chunker should expose its configuration:

```
{
  "max_chunk_tokens": 900,
  "target_chunk_tokens": 700,
  "min_chunk_tokens": 250,
  "allow_overlap": false,
  "respect_headings": true,
  "respect_speaker_turns": true,
  "break_on_topic_shift": true,
  "sentence_splitter": "spacy_sentencizer"
}
```

10.2 Chunk Boundary Algorithm

A simple deterministic chunking algorithm:

```

Initialize empty chunk.
For each sentence in source order:
  Estimate tokens for sentence.
  If adding sentence would exceed max_chunk_tokens:
    Close current chunk.
    Start new chunk.
  Add sentence to current chunk.
Close final chunk.

```

A more semantic algorithm:

```

For each sentence:
  Track token count.
  Track heading changes.
  Track speaker changes.
  Track paragraph boundaries.
  Track model or heuristic break signals.

When token count approaches target:
  Prefer closing at paragraph boundary.
  Otherwise prefer closing at heading boundary.
  Otherwise prefer closing before a strong semantic shift.
  Otherwise close before max token limit.

```

Julia Reader can start simple and become more sophisticated over time. What matters is that chunking is recorded, reproducible, and inspectable.

10.3 Overlap

Many RAG systems use overlapping chunks to preserve context across boundaries. Julia Reader can support overlap, but overlap should be treated carefully because it complicates coverage and claim accounting.

A default approach:

- no overlap for sentence coverage accounting,
- optional short context preview for model calls,
- clear distinction between **owned sentences** and **context sentences**.

Example:

```

{
  "chunk_id": "chunk_018",
  "owned_sentence_ids": ["s_000401", "...", "s_000432"],
  "context_before_sentence_ids": ["s_000396", "...", "s_000400"],
  "context_after_sentence_ids": ["s_000433", "...", "s_000436"]
}

```

This prevents duplicated context from being mistaken for duplicated source coverage.

11 Semantic Break Detection

Break detection identifies meaningful boundaries in the source.

A break may happen when:

- a new topic starts,
- a speaker changes the agenda,
- a document moves to a new section,
- a claim introduces a new argument,
- a story changes time or location,
- a technical explanation changes subsystem,
- a meeting moves from discussion to decision,
- a transcript changes from context to action items.

Break detection can be deterministic, model-backed, or hybrid.

11.1 Deterministic Signals

Deterministic signals include:

- headings,
- Markdown section markers,
- page breaks,
- large paragraph gaps,
- transcript timestamps,
- speaker turns,
- bullet-list starts and ends,
- token thresholds,
- section numbering,
- repeated agenda phrases.

11.2 Model-Backed Signals

Model-backed break detection can identify subtler shifts:

```
{
  "chunk_id": "chunk_018",
  "break_before": true,
  "break_after": false,
  "break_strength": 0.84,
  "break_reason": "The source moves from a high-level thesis into implementation
architecture.",
  "previous_topic": "project positioning",
  "new_topic": "core reading loop and lake design"
}
```

11.3 Why Breaks Matter

Break signals help the system:

- build better chunks,
- create chapter boundaries,
- preserve topic transitions,
- avoid flattening the source,
- support human browsing,
- and validate whether major source sections are represented.

Break detection should not be over-trusted. It is a signal, not an oracle. The lake should record break confidence and break reasons so they can be inspected.

12 Live Understanding

Live understanding is the running synthesis maintained during reading.

Instead of waiting until the end to summarize, Julia Reader keeps an evolving state:

```
What has been established?
What is the source about so far?
What entities matter?
What themes are emerging?
What claims have support?
What remains uncertain?
What changed in this chunk?
What should the reader watch for later?
```

A live understanding record may look like this:

```
{
  "state_id": "state_after_chunk_018",
  "run_id": "run_2026_05_09_001",
```

```

    "after_chunk_id": "chunk_018",
    "established_points": [
      {
        "point": "Julia Reader frames machine reading as a staged protocol rather than a single
prompt.",
        "support": ["s_000012", "s_000013", "s_000118"]
      }
    ],
    "current_themes": [
      "agentic reading",
      "source traceability",
      "progressive synthesis",
      "validation"
    ],
    "important_entities": [
      {
        "name": "Julia Reader",
        "type": "project",
        "notes": "Open-source research harness for agentic reading."
      }
    ],
    "open_questions": [
      "Which validation checks should be deterministic in the first release?",
      "How should the packet schema support downstream agent systems?"
    ],
    "recent_changes": [
      "The source moved from positioning into the architecture of the reading loop."
    ],
    "warnings": [],
    "version": 18
  }

```

12.1 Why Live Understanding Exists

Human reading is not usually a single compression operation at the end. We form an understanding as we go, then revise it as new material appears.

Julia Reader borrows this pattern.

The live understanding helps the model:

- maintain continuity across chunks,
- avoid treating each chunk as isolated,
- notice when later material revises earlier material,
- preserve open questions,
- and provide a compact state for later chunks without resending the entire source.

12.2 Live Understanding Is Not Hidden Chain of Thought

Live understanding should not be private hidden reasoning. It should be an explicit, user-inspectable state object.

It should contain grounded synthesis, not opaque scratchpad reasoning. It should say what the system currently believes the source has established and where that belief came from.

12.3 Update Contract

Each chunk-level read should produce an update:

```
{
  "chunk_id": "chunk_018",
  "local_summary": "This chunk explains the core reading loop and distinguishes deterministic
limbs from model-backed limbs.",
  "new_points": [],
  "revised_points": [],
  "confirmed_points": [],
  "contradictions_or_tensions": [],
  "new_entities": [],
  "new_themes": ["core loop", "lake metadata"],
  "open_questions": [],
  "suggested_chapter_role": "technical architecture"
}
```

The update is then merged into the latest live understanding.

13 The Lake

The lake is one of the most important parts of Julia Reader.

It is the structured evidence layer that makes the reading process durable, inspectable, recoverable, and useful beyond the final Markdown output.

A simple definition:

The lake is where Julia Reader stores the facts of the reading run.

The lake records:

- the source sentence map,
- the chunk map,
- break signals,
- reading events,
- local summaries,
- live-understanding history,
- extracted entities,
- themes,
- claims,
- claim support,

- chapter mappings,
- validation metadata,
- and export-ready packet fields.

The lake prevents the Chronicle from becoming a floating summary with no source memory.

13.1 Lake Design Principles

13.1.1 SOURCE FIRST

The source is the root of the system. Every generated artifact should be traceable back to the source through sentence IDs, chunk IDs, or source anchors.

13.1.2 IDS BEFORE PROSE

Generated prose can be rewritten. IDs should remain stable for a run. The system should assign IDs before asking a model to synthesize.

13.1.3 APPEND-MOSTLY HISTORY

The lake should prefer append-mostly logs for reading events and state history. The latest state can be overwritten for convenience, but the history should remain available.

13.1.4 JSONL FOR STREAMS

JSONL is useful for records that are appended one at a time, such as sentence maps, chunk summaries, events, and claims. It is simple to inspect, stream, and recover.

13.1.5 SEPARATE EVIDENCE FROM RENDERING

The lake stores evidence. The Chronicle renders that evidence into a human-readable book. The two should not collapse into one file.

13.1.6 RECOVERY IS A FIRST-CLASS GOAL

If a run fails at chunk 40 of 100, the system should know which chunks are complete and which remain pending.

13.1.7 VALIDATION READS THE LAKE

Validation should not depend only on the final Chronicle. It should inspect the lake to determine coverage, support, missing chunks, unsupported claims, and contradictions.

13.2 Lake Folder Layout

A practical lake layout:

```
lake/  
  manifest.json  
  sentence_map.jsonl  
  chunk_map.jsonl  
  break_map.jsonl  
  reading_events.jsonl  
  chunk_summaries.jsonl  
  live_understanding_history.jsonl
```

```
live_understanding_latest.json
entities.jsonl
themes.jsonl
claims.jsonl
claim_support.jsonl
chapter_map.jsonl
coverage.json
warnings.jsonl
```

For larger projects, the lake can be split into subfolders:

```
lake/
  maps/
    sentence_map.jsonl
    chunk_map.jsonl
    break_map.jsonl

  events/
    reading_events.jsonl
    warnings.jsonl

  state/
    live_understanding_history.jsonl
    live_understanding_latest.json

  extracts/
    chunk_summaries.jsonl
    entities.jsonl
    themes.jsonl
    claims.jsonl
    claim_support.jsonl

  artifacts/
    chapter_map.jsonl
    outline.json

  validation/
    coverage.json
    unsupported_claims.jsonl
    contradiction_checks.jsonl
```

The project should choose a default layout that is easy for beginners while still supporting growth.

13.3 Lake Record Types

13.3.1 MANIFEST

```
{
  "run_id": "run_2026_05_09_001",
  "chronicle_id": "chronicle_2026_05_09_001",
  "source_id": "src_2026_05_09_001",
  "created_at": "2026-05-09T23:00:00Z",
  "julia_reader_version": "0.2.0",
  "config": {
    "max_chunk_tokens": 900,
    "model_provider": "openai",
    "sentence_splitter": "spacy_sentencizer",
    "validation_level": "standard"
  },
  "status": "completed"
}
```

13.3.2 SENTENCE MAP

```
{
  "sentence_id": "s_000421",
  "text": "The project treats reading as an agent loop.",
  "position": 421,
  "chunk_id": "chunk_018",
  "estimated_tokens": 11,
  "source_anchor": {
    "page": null,
    "timestamp_start": null,
    "paragraph_id": "p_0081"
  }
}
```

13.3.3 CHUNK MAP

```
{
  "chunk_id": "chunk_018",
  "position": 18,
  "sentence_start": "s_000401",
  "sentence_end": "s_000432",
  "estimated_tokens": 742,
  "status": "processed"
}
```

13.3.4 READING EVENT

```
{
  "event_id": "event_000018",
  "event_type": "chunk_read",
  "chunk_id": "chunk_018",
  "started_at": "2026-05-09T23:06:10Z",
  "completed_at": "2026-05-09T23:06:22Z",
  "model": "reader-model-name",
  "input_tokens_estimated": 1430,
  "output_tokens_estimated": 612,
  "status": "completed",
  "warnings": []
}
```

13.3.5 CLAIM RECORD

```
{
  "claim_id": "claim_000093",
  "claim": "Julia Reader separates deterministic limbs from model-backed limbs.",
  "claim_type": "architecture",
  "support_sentence_ids": ["s_000602", "s_000603", "s_000604"],
  "support_chunk_ids": ["chunk_026"],
  "confidence": "medium",
  "generated_from": "chunk_026",
  "appears_in_chapter": "03-technical-architecture.md"
}
```

13.3.6 CHAPTER MAP

```
{
  "chapter_id": "chapter_03",
  "title": "Technical Architecture",
  "chapter_file": "chapters/03-technical-architecture.md",
  "source_chunk_ids": ["chunk_018", "chunk_019", "chunk_020", "chunk_021"],
  "source_sentence_ids": ["s_000401", "...", "s_000512"],
  "main_themes": ["core loop", "lake", "validation"],
  "status": "rendered"
}
```

13.4 The Lake as a Run Log

The lake should not only store extracted content. It should store the run itself.

Useful run information includes:

- when each chunk was processed,
- which model or backend processed it,
- which prompt version was used,
- whether retries occurred,

- whether a chunk failed,
- whether output passed schema validation,
- whether warnings were generated,
- whether validation later flagged the result.

This turns the lake into both an evidence store and an operations log.

13.5 The Lake Is Not the Chronicle

The Chronicle is for reading. The lake is for evidence.

A Chronicle chapter might say:

The source argues that machine reading should be treated as a protocol rather than a single prompt.

The lake should preserve the supporting claim record and sentence IDs.

This distinction matters because a Chronicle can be rewritten for different audiences while the lake remains the stable evidence base.

13.6 Lake and Resumability

Resumability should be built into the lake.

Each chunk can have a status:

```
pending
processing
processed
failed
skipped
needs_review
```

If the system fails, it can inspect `chunk_map.jsonl` and `reading_events.jsonl` to determine where to resume.

A simple recovery algorithm:

```
Load chunk map.
Load reading events.
Mark chunks with completed events as processed.
Mark chunks with failed events as failed.
Resume at first pending or failed chunk depending on config.
Do not rewrite completed chunk summaries unless rerun is explicit.
```

13.7 Lake and Human Review

The lake should support reviewer workflows.

A reviewer may ask:

- Which source chunks generated this chapter?
- Which claims lack sentence support?
- Which chunks were processed with warnings?
- Which sentences never appear in any chapter map?
- Which generated claims rely on low-confidence support?
- Which sections were compressed most aggressively?

The lake makes those questions answerable.

14 Local Reading

Each bounded chunk is processed through a local reading call.

The local reader should not be asked to produce a final book chapter. It should perform a focused task:

Read this chunk in light of the current live understanding.
 Identify what is new, what is important, what is supported, what changed, and what should be remembered.
 Return structured fields.

A local read result can contain:

```
{
  "chunk_id": "chunk_018",
  "local_summary": "The chunk explains Julia Reader's staged reading loop and introduces the lake as evidence storage.",
  "key_points": [
    {
      "point": "The reading loop separates source mapping, chunking, live synthesis, Chronicle generation, and validation.",
      "support_sentence_ids": ["s_000401", "s_000402", "s_000403"]
    }
  ],
  "entities": [
    {
      "name": "Julia Reader",
      "type": "project",
      "support_sentence_ids": ["s_000401"]
    }
  ],
  "themes": ["core loop", "lake metadata", "validation"],
  "open_questions": [
```

```

    "How should validation distinguish unsupported claims from high-level synthesis?"
  ],
  "contradictions_or_tensions": [],
  "chapter_suggestions": [
    {
      "suggested_title": "The Core Reading Loop",
      "reason": "This chunk explains the main staged process."
    }
  ]
}

```

14.1 Local Reader Prompt Contract

The prompt contract should require:

- no unsupported claims,
- use of sentence IDs when possible,
- separation of summary from interpretation,
- explicit uncertainty,
- extraction of open questions,
- preservation of source terminology,
- no final–chapter prose unless requested.

The output should be schema–validated. If the model returns malformed JSON, the system should repair or retry in a controlled way.

14.2 Local Read Is Not Final Synthesis

A local chunk summary is not the Chronicle. It is one piece of evidence.

The Chronicle is created later after all chunks have been processed and the lake contains enough context to build a global structure.

This distinction prevents early chunks from over–defining the final artifact.

15 Live Synthesis and State Updates

After each local read, Julia Reader updates the live understanding.

The update operation should be conservative. It should not rewrite the whole state unless necessary. It should record what changed.

A merge operation may follow these rules:


```

For each new point:
  If it supports an existing point, attach support.
  If it modifies an existing point, create a revision note.
  If it contradicts an existing point, create a tension record.
  If it is new and important, add it.
  If it is local detail only, store in chunk summary but not global state.

```

The update should distinguish:

- **established points** — source-supported claims that appear important globally,
- **local details** — details useful for a chapter but not global state,
- **open questions** — unresolved issues,
- **tensions** — contradictions or competing interpretations,
- **watch items** — things later chunks may clarify.

15.1 State Growth Control

Live understanding can become too large. Julia Reader needs state compaction.

A state compaction strategy:

```

Keep global thesis short.
Keep top themes.
Keep active entities.
Keep unresolved questions.
Keep recent changes.
Move older detailed points into lake records.
Preserve support IDs for important points.

```

The live state should be small enough to pass into later model calls, while the lake preserves full history.

15.2 State Versioning

Every live understanding update should have a version.

```

{
  "state_version": 19,
  "previous_state_version": 18,
  "after_chunk_id": "chunk_019",
  "change_summary": "Added validation as a first-class stage and connected it to claim support.",
  "state_hash": "sha256:..."
}

```

This allows the system to inspect how understanding evolved over time.

Chronicle Generation

The Chronicle is the core human-facing artifact.

It is a Markdown-based book generated from the source material and grounded in the lake.

A Chronicle may contain:

```
/index.md
/chapters/01-opening-context.md
/chapters/02-core-argument.md
/chapters/03-technical-architecture.md
/chapters/04-the-reading-lake.md
/chapters/05-validation-and-evaluation.md
/packets/reader_packet.json
/validation/validation_report.md
/lake/sentence_map.jsonl
/lake/chunk_map.jsonl
```

16.1 Chronicle Is Reconstruction, Not Compression Alone

The Chronicle should not simply compress the source. It should reconstruct it into a browsable form.

A good Chronicle:

- preserves the source's major structure,
- makes the material easier to navigate,
- uses chapter-level organization,
- surfaces themes and open questions,
- cites source sentences or chunks when appropriate,
- distinguishes source-grounded content from generated interpretation,
- and includes validation warnings.

16.2 Building the Outline

The outline builder can use:

- break map,
- chunk summaries,
- live understanding history,
- topic tags,
- entity map,
- source headings,
- source order,
- chapter suggestions from local reads,
- user preferences.

An outline record:

```
{
  "outline_id": "outline_001",
  "chapters": [
    {
      "chapter_id": "chapter_01",
      "title": "Why Long Context Is Not Enough",
      "source_chunk_ids": ["chunk_001", "chunk_002", "chunk_003"],
      "purpose": "Explain the problem Julia Reader addresses."
    },
    {
      "chapter_id": "chapter_02",
      "title": "The Core Reading Loop",
      "source_chunk_ids": ["chunk_004", "chunk_005", "chunk_006"],
      "purpose": "Explain the staged process from source to Chronicle."
    }
  ]
}
```

16.3 Chapter Rendering

A chapter renderer should receive:

- chapter title,
- intended audience,
- source chunks assigned to the chapter,
- relevant live understanding excerpts,
- claim records,
- source support IDs,
- style rules,
- validation constraints.

It should produce Markdown, not hidden prose.

A chapter can include source-aware notes:

```
## The Core Reading Loop
```

Julia Reader treats reading as a sequence of explicit operations. The source is first mapped into sentences, then grouped into bounded chunks, then processed through chunk-level reading events. Each event updates live understanding and writes evidence into the lake.

```
_Source support: chunks `chunk_004`-`chunk_006`; sentences `s_000091`-`s_000174`._
```

16.4 Multiple Chronicle Styles

The same lake can support different Chronicle styles:

```
technical_whitepaper
executive_brief
study_guide
meeting_digest
legal_review_packet
research_notes
developer_docs
```

The source evidence stays stable. The rendering changes.

17 Reader Packet

The packet is a downstream handoff object.

It is meant for agents, RAG systems, compliance tools, education tools, research assistants, report generators, or internal knowledge systems.

A packet should be compact but useful.

Example:

```
{
  "packet_version": "0.1",
  "source_title": "Meeting Transcript",
  "chronicle_id": "chronicle_2026_05_09_001",
  "run_id": "run_2026_05_09_001",
  "created_at": "2026-05-09T23:30:00Z",
  "summary": "The source explains Julia Reader, an open-source harness for agentic reading that maps sentences, chunks source material, updates live understanding, stores evidence in a lake, generates a Chronicle, and validates the result.",
  "themes": [
    "agentic reading",
    "sentence-level traceability",
    "reading lake",
    "Chronicle generation",
    "validation"
  ],
  "entities": [
    {
      "name": "Julia Reader",
      "type": "project"
    }
  ],
  "chapter_index": [
    {
      "title": "The Core Reading Loop",
      "file": "chronicle/chapters/02-core-reading-loop.md"
    }
  ],
  "open_questions": [
    "Which validation checks should ship in the first public release?"
  ]
}
```

```
],  
  "validation_status": "passed_with_warnings",  
  "lake_paths": {  
    "sentence_map": "lake/sentence_map.jsonl",  
    "chunk_map": "lake/chunk_map.jsonl",  
    "claims": "lake/claims.jsonl"  
  }  
}
```

17.1 Packet Design Goals

The packet should be:

- small enough for downstream use,
- structured enough for agents,
- source-aware enough for audit,
- independent of the full Chronicle prose,
- and versioned so future packet formats can evolve.

17.2 Packet Use Cases

A packet can support:

- indexing in a vector database,
- import into a knowledge graph,
- handoff to an agent planner,
- generation of a report,
- legal or compliance review,
- educational study materials,
- team memory,
- recurring project summaries.

18 Validation

Validation is the discipline layer.

A validation pass checks whether the reading run is internally consistent, source-grounded, and complete enough for its intended use.

Validation does not guarantee truth. It reduces silent failure.

18.1 Validation Questions

A validation system should ask:

1. Did every source sentence appear in the sentence map?
2. Did every sentence belong to exactly one owned chunk?
3. Were all chunks processed?
4. Did every processed chunk produce a reading event?
5. Did every chapter map back to source chunks?
6. Did every major generated claim have support?
7. Did any generated claim contradict the source or another claim?
8. Did the Chronicle omit entire source sections?
9. Did the system overstate uncertain evidence?
10. Were there errors, retries, or malformed outputs?

18.2 Coverage Validation

Coverage checks whether the run processed the source.

```
total_sentences = count(sentence_map)
owned_sentences = count(unique(sentence_ids in chunk_map))
processed_chunks = count(chunks with completed reading events)
total_chunks = count(chunk_map)

sentence_coverage = owned_sentences / total_sentences
chunk_coverage = processed_chunks / total_chunks
```

The validation report can include:

```
{
  "total_sentences": 1824,
  "sentences_assigned_to_chunks": 1824,
  "sentence_coverage": 1.0,
  "total_chunks": 83,
  "processed_chunks": 83,
  "chunk_coverage": 1.0,
  "missing_sentence_ids": [],
  "unprocessed_chunk_ids": []
}
```

18.3 Claim Support Validation

Claim support validation checks whether generated claims map back to source evidence.

A claim can be classified as:

```
supported
weakly_supported
unsupported
interpretive
needs_human_review
```

A claim does not need to be purely extractive. But the system should distinguish source-supported claims from interpretation.

Example validation output:

```
{
  "claim_id": "claim_000093",
  "claim": "Julia Reader is designed as a reading protocol rather than a single prompt.",
  "status": "supported",
  "support_sentence_ids": ["s_000042", "s_000043"],
  "notes": "The source explicitly states this distinction."
}
```

18.4 Contradiction and Tension Detection

Contradiction detection can compare:

- claims against source chunks,
- chapter text against claim map,
- early live understanding against later live understanding,
- local summaries against final Chronicle prose.

The output should not hide ambiguity. It should surface tensions.

```
{
  "tension_id": "tension_0007",
  "type": "possible_overstatement",
  "generated_text": "The validation system guarantees faithful reading.",
  "source_support": ["s_000812", "s_000813"],
  "issue": "The source says validation reduces silent failure, not that it guarantees faithful reading.",
  "recommended_fix": "Change 'guarantees' to 'attempts to improve'."
}
```

18.5 Validation Report

A Markdown validation report should be easy to read:

```
# Validation Report

## Status

Passed with warnings.
```

```
## Coverage

- Source sentences: 1,824
- Sentences assigned to chunks: 1,824
- Sentence coverage: 100%
- Chunks: 83
- Processed chunks: 83
- Chunk coverage: 100%

## Claim Support

- Claims checked: 219
- Supported: 198
- Weakly supported: 14
- Unsupported: 3
- Interpretive: 4

## Warnings

1. Chapter 4 contains a high-level synthesis with weak sentence-level support.
2. Claim `claim_00142` may overstate the source.
3. The source has no explicit answer for two open questions listed in the packet.
```

19

Deterministic and Model-Backed Limbs

Julia Reader can be understood as a set of deterministic and model-backed limbs.

19.1 Deterministic Limbs

Deterministic limbs should not depend on model creativity.

Examples:

- loading files,
- normalizing text,
- sentence ID generation,
- token estimation,
- chunk boundary enforcement,
- file writing,
- manifest creation,
- JSON schema validation,
- coverage accounting,
- recovery status checks,
- validation counts,
- deterministic export.

These limbs provide structure and safety.

19.2 Model-Backed Limbs

Model-backed limbs are used when synthesis or interpretation is required.

Examples:

- topic labeling,
- local chunk summary,
- entity extraction,
- semantic break interpretation,
- live understanding update,
- chapter title generation,
- open question extraction,
- contradiction detection,
- final chapter prose.

These limbs should be inspectable and replaceable. The project should allow developers to test different prompts, models, and strategies.

19.3 Why the Distinction Matters

The deterministic limbs create the spine. The model-backed limbs add interpretation.

If the two are blurred, the system becomes harder to debug. If a chunk is missing, that is not a model problem. It is a deterministic coverage problem. If a chapter overstates the evidence, that may be a model-backed synthesis problem caught by validation.

A good architecture lets contributors improve one limb without breaking all others.

20 Prompt and Schema Contracts

Julia Reader should treat prompts as versioned components, not casual strings.

Each model-backed limb should have:

- a prompt name,
- a prompt version,
- expected input schema,
- expected output schema,
- failure behavior,
- validation rules,
- and test examples.

Example:

```
{
  "prompt_name": "chunk_reader",
  "prompt_version": "0.1.0",
  "input_schema": "ChunkReaderInput",
  "output_schema": "ChunkReaderOutput",
  "required_fields": [
    "local_summary",
    "key_points",
    "entities",
    "themes",
    "open_questions"
  ],
  "failure_policy": "retry_then_mark_needs_review"
}
```

20.1 Why Schemas Matter

Without structured output, the lake becomes difficult to trust.

Schemas help the system:

- validate model outputs,
- retry malformed responses,
- compare model providers,
- run tests,
- build stable packets,
- and support downstream integrations.

20.2 Prompt Replacement

The project should encourage prompt experiments. A contributor should be able to replace `chunk_reader_v0.1` with `chunk_reader_v0.2` and compare outputs.

The lake manifest should record which prompt version produced each record.

21 Failure Modes

Julia Reader should be explicit about its failure modes.

21.1 Sentence Splitting Errors

A bad sentence split can damage traceability. The system should preserve character offsets and allow sentence-map inspection.

21.2 Bad Chunk Boundaries

A chunk may split an important idea. The system can reduce this by respecting headings, paragraphs, speaker turns, and semantic break signals.

21.3 Over-Compression

Local summaries may remove details needed later. The lake should preserve source sentence IDs and allow final stages to revisit source chunks.

21.4 State Drift

Live understanding may gradually drift. Julia Reader should periodically compare live state against accumulated source evidence and flag unsupported global claims.

21.5 Unsupported Chronicle Claims

The Chronicle may include claims not supported by source records. Claim support validation should catch this.

21.6 False Confidence

A polished Chronicle may sound authoritative even when validation warnings exist. The Chronicle should surface validation status clearly.

21.7 Model Variance

Different models may produce different summaries, themes, and chapter structures. Julia Reader should record model and prompt versions so runs can be compared.

21.8 Source Extraction Errors

PDF or OCR extraction may introduce errors. The loader should record extraction method and warnings.

21.9 Privacy and Sensitive Data

Long documents may contain sensitive information. The project should support local model modes, redaction hooks, and clear handling of source storage.

22 Evaluation Strategy

To become a serious research project, Julia Reader should include evaluation.

22.1 Source Coverage

Measure whether every sentence, paragraph, or chunk was processed and represented.

```
coverage = processed_sentences / total_sentences
```

22.2 Claim Traceability

Check whether generated claims map back to source sentence IDs.

Metrics:

```
traceable_claim_rate = claims_with_support / total_claims  
unsupported_claim_rate = unsupported_claims / total_claims
```

22.3 Summary Faithfulness

Compare local summaries and Chronicle chapters against source chunks for contradiction, omission, or unsupported claims.

22.4 Middle Retention

Create test documents where key facts appear at the beginning, middle, and end. Compare whether Julia Reader preserves those facts better than one-shot prompting.

This directly responds to the long-context reliability problem identified by “Lost in the Middle.” [1](#)

22.5 Human Review

Ask human reviewers whether the Chronicle is easier to inspect than:

- a flat summary,
- raw source text,
- a RAG answer with citations,
- or a map-reduce summary.

22.6 Downstream Usefulness

Test whether downstream agents perform better when given a Chronicle packet instead of raw text or a flat summary.

Possible tasks:

- answer questions about the source,
- produce a project plan,
- identify decisions and action items,
- create a study guide,
- extract compliance risks,
- compare multiple documents.

22.7 Resumability and Robustness

Test whether a run can fail midway and resume without corrupting the lake.

Metrics:

```
resume_success_rate
failed_chunk_recovery_rate
duplicate_record_rate
schema_error_rate
```

22.8 Latency and Cost

Measure:

- number of model calls,
- input tokens,
- output tokens,
- runtime,
- cost by provider,
- cost per source token,
- validation overhead.

The goal is not always to minimize cost. The goal is to make cost visible.

23 Example Use Case: Transcript to Chronicle

A user loads a long meeting transcript.

Julia Reader processes it as follows:

1. Source loading
The transcript is loaded, normalized, and assigned a source ID.
2. Sentence map
Every sentence receives an ID, with speaker and timestamp anchors when available.
3. Chunking
The transcript is divided into bounded semantic chunks.
4. Break detection
Agenda shifts and topic changes are detected.
5. Local reading
Each chunk is read in order.
6. Live understanding

A running synthesis updates after each chunk.

7. Lake writes

Sentence maps, chunk maps, summaries, entities, claims, and reading events are saved.

8. Chronicle

The meeting becomes a Markdown book with chapters.

9. Packet

Key decisions, open questions, entities, themes, and action items are saved.

10. Validation

The system checks for missing sections and unsupported claims.

The result is not merely:

Here is a summary of the meeting.

The result is:

Here is a structured reading artifact of the meeting.

Here are the chapters.

Here are the source maps.

Here are the claims.

Here are the unresolved questions.

Here is the validation status.

24 Example Use Case: Research Paper to Study Chronicle

A researcher loads a dense academic paper.

Julia Reader can produce:

- a chapter-by-chapter explanation,
- a list of key claims,
- a methods section breakdown,
- definitions of important terms,
- a claim-to-source map,
- unresolved questions,
- limitations,
- and a study packet.

The lake preserves where each explanation came from. This allows the researcher to inspect whether the Chronicle accurately represents the paper.

25 Example Use Case: Policy Document to Compliance Packet

A compliance team loads a policy document.

Julia Reader can produce:

- a readable policy Chronicle,
- obligation maps,
- risk areas,
- exceptions,
- definitions,
- responsible parties,
- unsupported or ambiguous interpretations,
- and a validation report.

For compliance, the lake is especially important because teams need source-grounded traceability. A summary is not enough. Reviewers need to know where a claim came from and whether any source sections were omitted.

26 Repository Structure

A suggested repository structure:

```
julia-reader/  
  README.md  
  LICENSE  
  CONTRIBUTING.md  
  SECURITY.md  
  
julia_reader/  
  __init__.py  
  cli.py  
  
core/  
  loader.py  
  normalizer.py  
  sentence_map.py
```

```
tokenizer.py
chunker.py
break_detector.py
live_understanding.py
chronicle_builder.py
packet_builder.py
validator.py
run_manager.py

lake/
  writer.py
  reader.py
  schemas.py
  manifest.py
  recovery.py

llm/
  base.py
  openai_provider.py
  local_provider.py
  prompts.py
  schema_validation.py

models/
  source.py
  sentence.py
  chunk.py
  break_signal.py
  reading_event.py
  live_state.py
  chronicle.py
  packet.py
  validation.py

utils/
  files.py
  ids.py
  logging.py
  hashing.py
  time.py

examples/
  sample_input.txt
  sample_chronicle/

tests/
  test_sentence_map.py
  test_chunker.py
  test_lake_writer.py
  test_validator.py
  test_recovery.py

docs/
  whitepaper.md
  concepts.md
  architecture.md
  lake.md
  validation.md
  roadmap.md
```


27

CLI Sketch

A simple CLI can make the project approachable.

```
julia-reader read examples/sample_input.txt \  
  --title "Sample Input" \  
  --out chronicles/sample_run \  
  --max-chunk-tokens 900 \  
  --chronicle-style technical_whitepaper \  
  --validate
```

Possible commands:

julia-reader read	Run the full reading loop.
julia-reader map	Build only the sentence map and chunk map.
julia-reader resume	Resume an interrupted run.
julia-reader validate	Run validation on an existing lake and Chronicle.
julia-reader render	Re-render a Chronicle from an existing lake.
julia-reader packet	Build or rebuild the downstream packet.
julia-reader inspect	Print run status, coverage, and warnings.

The CLI should make the stages visible rather than hiding them.

28

Open-Source Philosophy

Julia Reader should be open-sourced not as a finished answer, but as an invitation.

The repo should tell developers:

We are exploring how machines should read. This is one harness. Fork it, challenge it, replace parts of it, and test better methods.

The most valuable community contributions may include:

- better sentence splitters,
- better chunking algorithms,
- better break detection,
- better local model prompts,
- better Chronicle formats,

- better validation methods,
- graph-based extensions,
- RAG integrations,
- UI visualizations,
- benchmark datasets,
- privacy-preserving local modes,
- and reproducible evaluation suites.

The project should welcome disagreement. The point is not to defend one final reading loop. The point is to make reading loops testable.

29 Roadmap

Phase 1: Public Research Release

Goals:

- clean README,
- MIT license,
- basic CLI,
- sample input,
- sample Chronicle output,
- clear install instructions,
- clear definitions,
- root `WHITE_PAPER.md`,
- docs mirror at `docs/whitepaper.md`.

Phase 2: Reading Loop Stability

Goals:

- robust sentence mapping,
- chunk metadata,
- lake writer,
- live understanding file,
- deterministic run logs,
- error recovery,
- reproducible output folders.

Phase 3: Chronicle System

Goals:

- index generation,
- chapter generation,
- page splitting,
- source references,
- Markdown output,
- packet export,
- multiple Chronicle styles.

Phase 4: Validation

Goals:

- source coverage checks,
- missing chunk detection,
- unsupported claim warnings,
- contradiction flags,
- validation report,
- validation JSON output.

Phase 5: Research Extensions

Goals:

- compare chunking strategies,
- support multiple LLM backends,
- add local model mode,
- run benchmark tests,
- publish examples,
- evaluate middle retention.

Phase 6: Agent Integrations

Goals:

- RAG export,
- LangChain/LlamaIndex adapters,
- downstream agent packet format,
- Chronicle-to-memory workflows,
- graph export,
- API endpoints.

30 What Julia Reader Is Not

Julia Reader should be clear about its limits.

It is not:

- a finished comprehension engine,
- a replacement for RAG,
- a replacement for human review,
- a guarantee against hallucination,
- a claim that LLMs understand like humans,
- a claim that chunking itself is novel,
- a magic solution to long-context failures,
- or a complete benchmark suite yet.

It is:

- an experimental harness,
- a reading protocol,
- a Chronicle generator,
- a structure-preserving preprocessing tool,
- a source-grounded artifact builder,
- and a research environment for testing agentic reading methods.

31 Why This Matters

31.1 For Agents

Agents increasingly need to process long materials: transcripts, codebases, contracts, research papers, strategy documents, ticket histories, and policy files.

Without structured reading, agents often rely on fragile context windows or shallow retrieval.

Julia Reader gives agents a way to create a durable intermediate object before acting.

31.2 For RAG

RAG systems often ingest chunks without enough narrative or semantic structure.

Julia Reader can become a preprocessing layer that turns raw material into richer, better-labeled, more navigable input.

Instead of:

```
raw PDF → chunks → vector database
```

Julia Reader enables:

```
raw PDF → Chronicle → sentence map → chapter map → packet → retrieval/indexing
```

31.3 For Compliance and Legal Workflows

In compliance, auditability matters.

A summary is not enough. Teams need to know:

- where a claim came from,
- whether all evidence was reviewed,
- what was omitted,
- and whether outputs are grounded in source material.

Julia Reader's sentence maps, metadata, and validation reports are useful for this kind of work.

31.4 For Education

Chronicle generation can help turn dense material into browsable learning resources.

A long lecture, transcript, or article can become:

- chapters,
- study notes,
- definitions,
- source-linked summaries,
- open questions,
- review packets.

31.5 For Research

Researchers often need to process large bodies of text while preserving nuance.

Julia Reader can support:

- literature review,
- interview analysis,
- meeting transcript analysis,
- qualitative coding,
- thematic mapping,
- source-grounded synthesis.

32

Short Public Thesis

Julia Reader explores whether agents can read better when long-form input is processed as a progressive, inspectable loop instead of a single prompt, flat summary, or hidden RAG index.

A slightly more technical version:

Julia Reader is an open research harness that maps source text into sentences, builds bounded chunks, reads them through a stateful loop, stores evidence in a lake, reconstructs the result as a Markdown Chronicle, and validates source coverage and claim support.

A builder-facing version:

Julia Reader gives agents a repeatable way to read long material before acting on it: map it, chunk it, read it, store the evidence, build a Chronicle, export a packet, and validate the result.

33

Conclusion

Julia Reader begins from a practical frustration: long-form input is still hard for LLMs. Even when the context window is large, models can lose the middle, drift from the original purpose, compress away important details, or produce fluent but poorly grounded summaries.

The answer is not only more context.

The answer may be better reading loops.

Julia Reader explores this possibility by treating reading as a staged, inspectable process. It combines deterministic structure with model-backed synthesis, preserves sentence-level metadata, updates live understanding as it moves through the source, stores evidence in a lake, reconstructs the material into a Chronicle, and validates the result before downstream use.

The project's purpose is not to declare a final method for machine reading. Its purpose is to create a shared harness where developers, researchers, and builders can test better ones.

Julia Reader is an open research project for making machine reading slower, more structured, more inspectable, and more useful.

A

APPENDIX A

Minimal Data Schemas

A.1 Sentence

```
class SentenceRecord(BaseModel):
    sentence_id: str
    source_id: str
    text: str
    position: int
    paragraph_id: str | None = None
    chunk_id: str | None = None
    char_start: int | None = None
    char_end: int | None = None
    estimated_tokens: int
    source_anchor: dict = {}
    flags: list[str] = []
```

A.2 Chunk

```
class ChunkRecord(BaseModel):
    chunk_id: str
    source_id: str
    position: int
    sentence_ids: list[str]
    sentence_start: str
    sentence_end: str
    estimated_tokens: int
    previous_chunk_id: str | None = None
    next_chunk_id: str | None = None
    break_before: bool = False
    break_after: bool = False
    topic_tags: list[str] = []
    status: str = "pending"
```

A.3 Reading Event

```
class ReadingEvent(BaseModel):
    event_id: str
    run_id: str
    chunk_id: str
    event_type: str = "chunk_read"
    started_at: str
    completed_at: str | None = None
    model_provider: str | None = None
    model_name: str | None = None
    prompt_version: str | None = None
    status: str
    warnings: list[str] = []
```

A.4 Claim

```
class ClaimRecord(BaseModel):
    claim_id: str
    claim: str
    claim_type: str
    generated_from_chunk_id: str
    support_sentence_ids: list[str] = []
    support_chunk_ids: list[str] = []
    confidence: str = "unknown"
    validation_status: str = "unchecked"
```

A.5 Packet

```
class ReaderPacket(BaseModel):
    packet_version: str
    source_title: str
    run_id: str
    chronicle_id: str
    summary: str
    themes: list[str]
    entities: list[dict]
    chapter_index: list[dict]
    open_questions: list[str]
    validation_status: str
    lake_paths: dict
```


B

APPENDIX B

Example Validation Rules

```

RULE: sentence_coverage_complete
PASS if every sentence_id in sentence_map appears in exactly one chunk's owned_sentence_ids.

RULE: all_chunks_processed
PASS if every chunk_id in chunk_map has at least one completed reading_event.

RULE: claim_has_support
WARN if a non-interpretive claim has no support_sentence_ids.

RULE: chapter_has_source_mapping
WARN if a chapter has no source_chunk_ids.

RULE: no_unprocessed_chunks_in_chronicle
FAIL if the Chronicle uses a chunk marked failed or unprocessed.

RULE: live_state_has_versions
PASS if every live understanding record has a monotonically increasing version.

```

C

APPENDIX C

Recommended README Positioning

```

# Julia Reader

Julia Reader is an experimental open-source harness for agentic reading.

It turns long-form source material into a structured Chronicle by mapping sentences, detecting
breaks, creating bounded chunks, refreshing a live understanding, storing evidence in a reading
lake, generating Markdown book-style outputs, preparing downstream packets, and validating the
run.

The project is based on a simple research belief:

> Longer context windows are useful, but they do not remove the need for structured reading.

Julia Reader is not just a summarizer and not just a RAG preprocessor. It is a place to
experiment with how agents can read progressively, preserve source structure, and produce
artifacts that humans and other systems can inspect.

```



APPENDIX D

Recommended Next Step for the Repo

This document can live at the repository root as `WHITE_PAPER.md` and as `docs/whitepaper.md`.

The Next.js demo can serve the same file at `/whitepaper` as a static asset, for example:

```
public/whitepaper.md
```

After editing the root file, from `demo/nextjs-reader` run:

```
npm run sync:whitepaper
```

Link from the main README under **Research white paper**.

● References

- [1] **Lost in the Middle: How Language Models Use Long Contexts (TACL)**
<https://aclanthology.org/2024.tacl-1.9/>
- [2] **Lost in the Middle: How Language Models Use Long Contexts**
<https://arxiv.org/abs/2307.03172>
- [3] **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks**
<https://arxiv.org/abs/2005.11401>
- [4] **Hierarchical node parsers — LlamaIndex**
https://developers.llamaindex.ai/python/framework-api-reference/node_parsers/hierarchical/
- [5] **MapReduceDocumentsChain — LangChain**
https://reference.langchain.com/python/langchain-classic/chains/combine_documents/map_reduce/MapReduceDocumentsChain
- [6] **Welcome — GraphRAG**
<https://microsoft.github.io/graphrag/>
- [7] **AutoGen**
<https://microsoft.github.io/autogen/stable/index.html>
- [8] **Sentencizer — spaCy API Documentation**
<https://spacy.io/api/sentencizer>